

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590018



PROJECT REPORT

on

“Using EEG Signals and Machine Learning to Improve the Early Detection of Major Depressive Disorder (MDD)”

Submitted in partial fulfilment for the award of the degree

Bachelor of Engineering

in

Computer Science and Engineering

Submitted by

**Veeresh K G
Shashank S V
Pramod N
Chethan R**

**1ST21CS232
1ST21CS191
1ST21CS154
1ST21CS055**

Under the Guidance of

**Prof. Poovidha S
Assistant Professor
Department of CSE**



SAMBHRAM
INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

2024-2025

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the Project work entitled “**Using EEG Signals and Machine Learning to Improve the Early Detection of Major Depressive Disorder (MDD)**” carried out by Mr. **Veeresh K G**, USN- 1ST21CS232, Mr. **Shashank S V**, USN- 1ST21CS191, Mr. **Pramod N**, USN- 1ST21CS154, Mr. **Chethan R**, USN- 1ST21CS055, bonafide student of **SAMBHRAM INSTITUTE OF TECHNOLOGY** in partial fulfilment for the award of **BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING** of **VISVESVARAYA TECHNOLOGICAL UNIVERSITY**, Belagavi during the year **2024-2025**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The Project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Prof. Poovidha S
Assistant Professor
Dept. of CSE
SaIT, Bengaluru

Dr. T. John Peter
HOD
Dept. of CSE
SaIT, Bengaluru

Dr. H. G. Chandrakanth
Principal
SaIT, Bengaluru

EXTERNAL VIVA:

Name of the Examiners

Signature with date

1.

2.

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

STUDENT DECLARATION

We, Mr. **Veeresh K G**, USN- 1ST21CS232, Mr. **Shashank S V**, USN- 1ST21CS191, Mr. **Pramod N**, USN- 1ST21CS154, Mr. **Chethan R**, USN- 1ST21CS055, hereby declare that the project work entitled “ **Using EEG Signals and Machine Learning to Improve the Early Detection of Major Depressive Disorder (MDD)**” submitted to **Visvesvaraya Technological University**, is a record of an original work done by me/us under the guidance of **Prof. Poovidha S, Assistant Professor** at **Department of Computer Science and Engineering, Sambhram Institute of Technology**.

We further declare that this project work has not been submitted previously for the award of any degree, diploma, or other academic recognition at any other institution. We have acknowledged all sources used in this project, and we understand that plagiarism is a serious offense.

Veeresh K G
Shashank S V
Pramod N
Chethan R

Date:

ABSTRACT

Major Depressive Disorder (MDD) is a common and serious mental health condition, yet its diagnosis remains largely subjective and relies on clinical assessments, leading to inconsistent outcomes and delayed treatment. This research project aims to address this gap by developing a sophisticated machine learning system for MDD detection using electroencephalography (EEG) data. The proposed system leverages advanced algorithms like Convolutional and Recurrent Neural Networks to analyse EEG signals, capturing minor anomalies in brain activity linked with MDD. The methodology involves data collection and preprocessing, followed by feature extraction using techniques like wavelet transforms, and model training. The performance is evaluated using metrics such as accuracy, precision, and recall. This approach tackles challenges like signal noise and limited data availability through innovative techniques such as data augmentation and feature engineering. The expected outcome is a robust system for accurately classifying individuals as MDD-positive or healthy, leading to early and accurate diagnosis. The broader societal impact will be to facilitate timely interventions, reduce burdens on healthcare systems and improve mental health care accessibility. This project moves forward the field of computational psychiatry and opens the door to better diagnostic tools.

ACKNOWLEDGEMENT

Any achievement, be it scholastic or otherwise does not depend solely on the individual efforts but on the guidance, encouragement and cooperation of intellectuals, elders and friends. A number of personalities, in their own capacities have helped us in carrying out this project work. We would like to take this opportunity to thank them all.

We would like to express our heartfelt thanks to **Dr. H. G. Chandrakanth, Principal, Sambhram Institute of Technology**, whose valuable guidance has been the one that helped us to complete the project.

We would like to express our profound gratitude to **Dr. T. John Peter, HOD, Department of CSE, Sambhram Institute of Technology**, for his suggestions and his instructions have served as the major contribution towards the completion of the project.

We would like to extend our impassioned thanks and admiration to our **guide, Prof. Poovidha S, Assistant Professor, Department of CSE, Sambhram Institute of Technology**, for their able guidance, regular source of encouragement and assistance throughout this project.

We would like to thank all the teaching and non-teaching staff members of the Computer Science Department, who have helped us directly or indirectly for the successful completion of the project.

Finally, we would like to thank our Parents and Friends who have helped us with their valuable suggestions and guidance for the completion of our project

Veeresh K G	1ST21CS232
Shashank S V	1ST21CS191
Pramod N	1ST21CS154
Chethan R	1ST21CS055

TABLE OF CONTENTS

Abstract	i
Acknowledgement	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi

Chapter No.	Title	Page No.
	INTRODUCTION	1
	1.1 Overview	1
	1.2 Problem statement	2
1	1.3 Scope of Project	2
	1.4 Objectives	3
	LITERATURE SURVEY	4
	2.1 Survey	4
2	2.2 Motivation	4
	2.3 Eeg-based classification for depression	5
	2.4 Deep learning approaches	5
	2.5 Feature Engineering and Channel Selection	5
	2.6 Challenges and Opportunities	5
	SYSTEM DESIGN	6
	3.1 System architecture	6
	3.2 Flow chart	14
3	3.3 Sequence diagram	15
	3.4 ICA component map	16
	3.5 Use cases	17
	SOFTWARE AND HARDWARE REQUIREMENTS	18
4	4.1 Software Requirements	18
	4.2 Hardware Requirements	19
	IMPLEMENTATION	20
	5.1 Mdd prediction system - App.py	20
5	5.2 Mdd prediction system – model prediction. ipynb	24
	5.3 Mdd prediction system – preprocessing. Ipynb	28
	5.4 Algorithm	29
	TESTING	32
6	6.1 Project Testing	32
	6.2 Dataset of healthys.csv for Testing	33

	RESULTS	34
7	7.1 Snapshots of Project	34
8	CONCLUSION	36
	FUTURE SCOPE	37
	REFERENCE	38

LIST OF FIGURES

Figure No.	Figure Label	Page No.
3.1	Architectural diagram	06
3.2	Hierarchical structure of data	07
3.3	Pipeline for preprocessing	08
3.4	Model Architecture	11
3.5	Flow chart of Proposed System	14
3.6	Sequence Diagram	15
3.7	ICA component map	16
4.1	Python	19
4.2	Machine Learning	19
4.3	Vite	19
4.4	HTML and CSS	19
5.1	Performance metrics of Logistic regression	30
5.2	performance metrics of Random Forest	30
5.3	Performance metrics of SVM	31
5.4	Performance metrics of XGBoost	31
6.1	Health dataset	33
7.1	User Interface	34
7.2	File Upload Interface	34
7.3	Predicting Person as healthy	35
7.4	Predicting Person as MDD	35

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Description of Files in Dataset	07
3.2	Extracted EEG Features and their Description	09
3.3	Models used for classification	10
5.1	Comparison Of Performance Measure	29

CHAPTER 01

INTRODUCTION

1.1 Overview

Major Depressive Disorder (MDD) is a debilitating mental health condition that affects millions worldwide, leading to a significant decline in quality of life, social functioning, and productivity. According to the World Health Organization (WHO), depression is one of the leading causes of disability globally. Individuals suffering from MDD experience persistent sadness, loss of interest or pleasure in daily activities, and a range of cognitive and physical symptoms such as fatigue, sleep disturbances, and difficulty concentrating. Despite its prevalence, the diagnosis of MDD remains largely subjective, primarily relying on clinical interviews and self-reported symptoms. This lack of objective diagnostic tools often results in inconsistent assessments, delayed diagnosis, and ineffective treatment plans. These factors highlight an urgent need for innovative approaches that can improve the accuracy and timeliness of MDD diagnosis.

Recent advancements in neuroscience and machine learning offer promising solutions to this diagnostic challenge. Electroencephalography (EEG), a non-invasive method for recording electrical activity in the brain, has emerged as a valuable tool for analyzing neural patterns associated with mental health disorders. EEG signals can reveal subtle changes in brain activity that may be indicative of MDD, even in its early stages. However, extracting meaningful insights from EEG data presents several challenges due to its complex and noisy nature. This is where machine learning algorithms, particularly deep learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), can play a transformative role. These models have shown remarkable success in capturing intricate patterns within time-series data, making them well-suited for EEG signal analysis.

This project aims to develop an advanced machine learning system for MDD detection by leveraging EEG data and deep learning techniques. The proposed system will focus on analyzing EEG signals to identify subtle anomalies linked to MDD, providing an objective and reliable diagnostic tool. The methodology involves several key steps: data collection and pre-processing, feature extraction using techniques like wavelet transforms, and model training and evaluation. The feature extraction process is critical for converting raw EEG data into meaningful representations that the machine learning models can effectively interpret.

1.2 Problem Statement

Major Depressive Disorder (MDD) is a prevalent and serious mental health condition that often goes undiagnosed or is diagnosed late due to its subjective symptoms and the social stigma associated with mental health issues. Traditional diagnostic methods largely depend on self-reported symptoms and clinical interviews, which can be inconsistent and time-consuming. Early detection of MDD is crucial for effective treatment and prevention of long-term psychological and physiological effects. However, existing diagnostic systems lack objective biomarkers for reliable and timely identification. Electroencephalography (EEG) signals offer a non-invasive, cost-effective, and objective means of assessing brain activity associated with depression. By leveraging machine learning techniques to analyze EEG data, it is possible to improve the accuracy and speed of MDD diagnosis.

1.3 Scope Of the Project

- **Early Detection of Major Depressive Disorder (MDD):** Develop a system that uses EEG signal data and machine learning techniques to identify early signs of MDD with high accuracy.
- **EEG Signal Processing:** Implement robust preprocessing techniques to clean and extract relevant features from raw EEG signals for improved model performance.
- **Machine Learning Model Training:** Train and optimize machine learning models using EEG datasets from diagnosed and healthy individuals to distinguish patterns indicative of MDD.
- **Feature Extraction and Selection:** Integrate methods to identify critical EEG biomarkers linked to depressive symptoms, improving diagnostic precision.
- **Scalability:** Build the system architecture to support the inclusion of additional mental health disorders and multimodal data
- **Data Privacy and Security:** Ensure patient EEG data is securely stored and processed, maintaining strict compliance with health data privacy standards.
- **Remote Healthcare Support:** Enable remote screening and assessment for MDD through integration with portable EEG devices, improving access for underserved populations.

1.4 Objectives

- To analyse EEG signal patterns associated with Major Depressive Disorder (MDD) and identify key biomarkers for early diagnosis.
- To preprocess and extract relevant features from raw EEG data to enhance signal quality and ensure accurate input for machine learning models.
- To develop and train machine learning algorithms that can classify EEG data to distinguish between healthy individuals and those with early signs of MDD.
- To evaluate the performance of various machine learning models (e.g., SVM, Random Forest, Neural Networks) in terms of accuracy, sensitivity, and specificity for early detection.
- To create a reliable and non-invasive diagnostic framework that leverages EEG and machine learning for practical, clinical use in mental health screening.
- To reduce diagnosis time and subjectivity in identifying MDD by supporting clinical decisions with data-driven tools.
- To implement advanced signal processing techniques (e.g., wavelet transform, FFT) for denoising and enhancing EEG signal clarity.
- To explore deep learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) for automatic feature learning and classification.
- To integrate domain knowledge from neuroscience and psychiatry to improve model interpretability and relevance in clinical settings.
- To develop a standardized pipeline for EEG data acquisition, preprocessing, and model training/testing to ensure reproducibility of results.

CHAPTER 02

LITERATURE SURVEY

2.1 Survey

The application of machine learning (ML) techniques to electroencephalogram (EEG) signals has gained significant attention in mental health research, particularly for the early detection of Major Depressive Disorder (MDD). EEG provides a non-invasive, cost-effective method for measuring brain activity, offering rich temporal data that can be analyzed to detect abnormalities associated with depression. Studies have demonstrated that ML algorithms, such as Support Vector Machines (SVM), Random Forests.

Recent advancements have focused on extracting meaningful features from EEG signals, such as power spectral density, entropy, and functional connectivity, to train ML models with high accuracy and sensitivity. Projects integrating EEG and ML show promise in supporting clinicians by providing objective tools for MDD diagnosis. Despite progress, challenges persist, including variability in EEG signals across individuals, lack of large and balanced datasets, and limited generalization across demographics. This project builds upon these studies by aiming to develop a scalable and accurate ML-based system that utilizes EEG features for early MDD detection, addressing issues of data variability and focusing on robust preprocessing and feature engineering strategies.

2.2 Motivation

The motivation for this project arises from the growing need for accurate and early detection of Major Depressive Disorder (MDD) using advanced artificial intelligence techniques. MDD is a leading cause of disability worldwide, and its diagnosis is often based on subjective clinical assessments, which can result in delays and inconsistencies in treatment. With advancements in computational psychiatry, integrating machine learning with electroencephalography (EEG) data offers a promising approach to improving diagnostic accuracy. EEG provides a non-invasive method to analyse brain activity, and machine learning models can effectively detect patterns associated with MDD. By leveraging techniques such as feature extraction, data augmentation, and classification algorithms, this project aims to build a robust system that can classify individuals as either MDD patients or health controls. The objective is to enhance early detection, facilitate timely intervention, and contribute to more reliable and scalable mental healthcare solutions.

2.3 EEG-Based Classification for Depression

Several studies have utilized machine learning models to classify EEG data for depression detection. Hosseini Fard et al. (2013) applied feature extraction techniques like wavelet transform and used classifiers such as SVM and k-NN, achieving high classification performance. Similarly, Mumtaz et al. (2017) applied a hybrid feature selection approach to improve the SVM model's ability to detect depression. These studies confirm the viability of EEG-based classification frameworks using ML for early MDD diagnosis.

2.4 Deep Learning Approaches

Recent developments have seen the application of deep learning, particularly CNNs and Long Short-Term Memory (LSTM) networks, to EEG-based depression detection. Zeng et al. (2019) employed a deep CNN model trained on EEG spectrograms, reporting superior performance compared to traditional ML models. Likewise, Zhang et al. (2020) used a hybrid CNN-LSTM model to capture both spatial and temporal EEG features, significantly enhancing depression classification accuracy. These studies underscore the potential of deep learning to automate feature extraction and improve classification outcomes.

2.5 Feature Engineering and Channel Selection

Optimizing EEG channel selection and feature engineering has been critical in improving model performance. Acharya et al. (2015) emphasized the importance of selecting relevant EEG channels and time-frequency features, showing that reduced channel sets could still yield high classification accuracy. Furthermore, Lee et al. (2011) demonstrated that alpha and theta band power in the frontal and parietal regions were strongly correlated with depressive symptoms, guiding feature selection in ML-based models. These findings show that intelligent preprocessing and feature optimization are key to effective EEG-based

2.6 Challenges and Opportunities

Despite promising results, several limitations exist in current EEG-ML systems for MDD detection. Data scarcity, class imbalance, and inter-subject variability often affect model generalizability. Mohammadi et al. (2019) highlighted the need for larger, multi-centre datasets and standardized EEG acquisition protocols. Additionally, interpretability of ML models remains a challenge, particularly in clinical settings. Future research must focus on explainable AI (XAI) approaches and rigorous validation to ensure trust and clinical adoption. This project aims to address these gaps by building a generalized and interpretable ML framework for early MDD detection using standardized EEG protocols.

SYSTEM DESIGN

3.1 System Architecture

Electroencephalography (EEG) is a powerful tool used to measure electrical activity in the brain, making it highly relevant for applications such as diagnosing neurological disorders, mental health assessments, and cognitive research. In the context of Major Depressive Disorder (MDD) or other cognitive conditions, EEG data can be leveraged in a structured machine learning pipeline to enhance early detection and analysis.

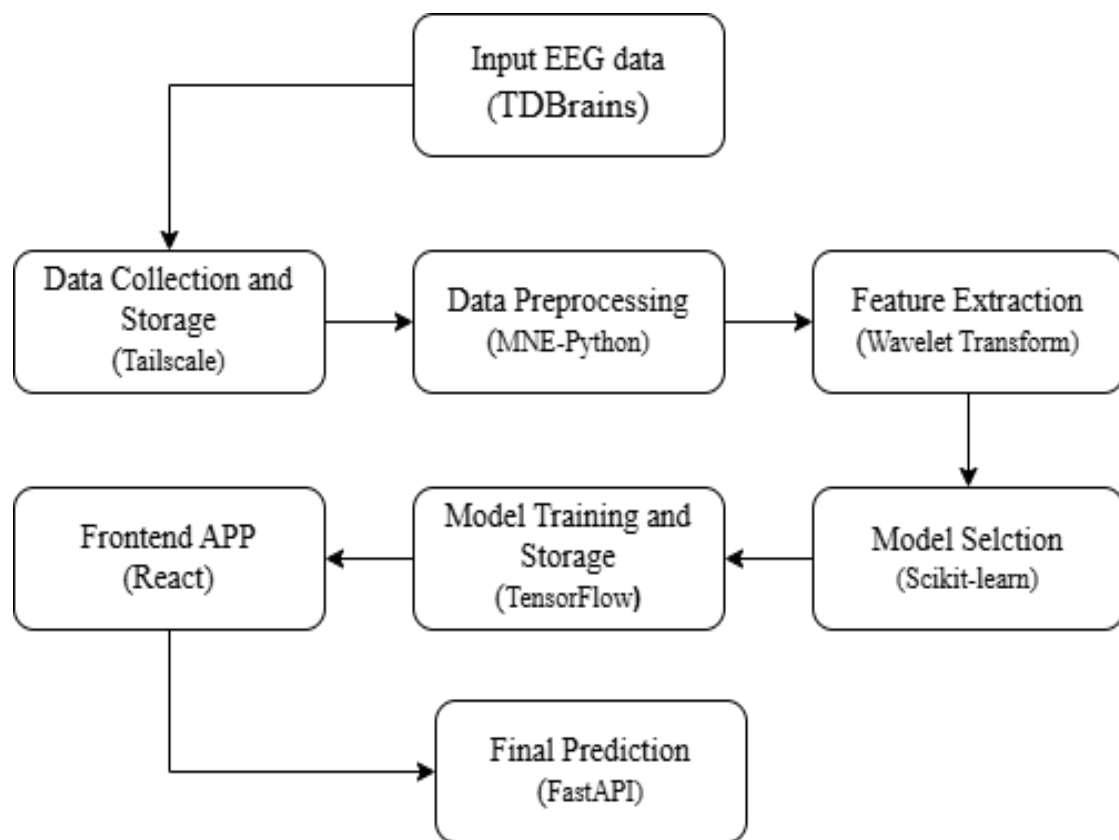


Fig no.3.1 Architectural diagram

The above Fig 3.1 provides a comprehensive explanation of the different stages involved in an EEG-based machine learning system, focusing on data processing, model training, and implementation through a user-friendly frontend application.

1. Input EEG Data

The system is acquiring raw EEG data from patients or subjects. EEG signals are captured using electrodes placed on the scalp, recording electrical brain activity over time. The signals are typically recorded in multiple

Using EEG Signals and Machine Learning to Improve the Early Detection of Major Depressive Disorder channels corresponding to different brain regions. This raw data forms the foundation for further processing and analysis. This data is obtained from TDBrains clinics dataset.

2. Data Collection and Storage

Once acquired, the EEG data needs to be systematically collected and stored. This step ensures that the data remains organized, accessible, and formatted correctly for future processing. The data is usually stored in structured databases, cloud storage, or local servers with proper indexing and metadata to facilitate efficient.



Fig no. 3.2 Hierarchical structure of data

The above Fig 3.2 illustrates hierarchical structure of data organizes data in a tree-like format with parent-child relationships, where each parent can have multiple children, but each child has only one parent.

TABLE 3.1. Description of Files in Dataset

File Name	Description
.eeg	The raw EEG data (binary format).
.vhdr	The header file containing metadata about the EEG recording (sampling rate, channel names, etc.).
.vmrk	The marker file that contains event markers (stimuli, blinks, artifacts, etc.).
.json	Metadata file containing subject-specific details.
.tsv	Tab-separated file with additional channel-specific metadata.

This table 3.1 describes EEG data files, where. Eeg stores raw brain signals, vhdr provides metadata like sampling rate, and. vmrk logs event markers such as stimuli. The .Json file contains subject-specific details, while. tsv includes channel metadata like electrode positions. together, these files ensure accurate EEG data processing and analysis.

3. Data Preprocessing

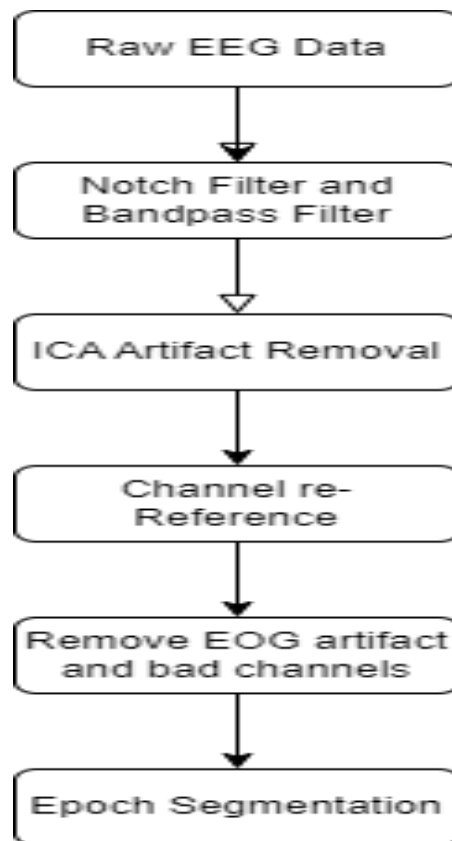


Fig no. 3.3 Pipeline for preprocessing

The above Fig 3.3 illustrates the systematic preprocessing pipeline used for EEG signal processing to enhance data quality and remove unwanted noise and artifacts. Preprocessing EEG signals is a critical step to ensure that the data used for analysis and classification is free from noise, artifacts, and irrelevant information. The raw EEG signals are often contaminated with various sources of noise, such as power line interference, eye movements, and muscle activity, which can significantly affect the accuracy of the final results. A structured approach is followed to improve the signal quality and ensure that meaningful neural activity is preserved.

The EEG signals are recorded from subjects using an EEG acquisition system equipped with multiple electrodes placed on the scalp. The placement of electrodes follows the 10-20 international system to ensure standardized positioning across participants. Each electrode captures electrical activity generated by neurons, but this raw signal also includes unwanted noise from external and physiological sources.

4. Feature Extraction

After preprocessing, the next step involves extracting meaningful features from the EEG data. These features represent the most relevant characteristics of brain activity, which help in distinguishing different mental states or conditions. Feature extraction methods can include time-domain analysis (e.g., statistical moments), frequency-domain analysis (e.g., power spectral density), and time-frequency representations (e.g., wavelet transforms). Selecting the right set of features improves model accuracy and interpretability.

Feature extraction in EEG analysis is a crucial step that involves transforming raw or preprocessed signals into meaningful representations that capture the most relevant aspects of brain activity. This process enhances the ability of machine learning models to classify different mental states, cognitive processes, or neurological conditions. Various feature extraction techniques are applied depending on the type of information required.

Time-domain features provide statistical descriptions of the EEG signal by analyzing its amplitude variations over time. Commonly used features include the mean, variance, skewness, kurtosis, and peak-to-peak amplitude, which help quantify signal fluctuations. Additionally, Hjorth parameters, such as activity, mobility, and complexity, offer insights into the signal's dynamic properties. These features are particularly useful for applications where temporal variations play a crucial role, such as event-related potential (ERP) analysis.

TABLE 3.2: Extracted EEG Features and their Description

Feature Type	Feature Description	
	Feature Name	Description
Time-Domain	Mean Amplitude	Average signal value over time.
	Variance	Measures signal fluctuation intensity.
	Skewness & Kurtosis	Describes the shape of the EEG signal distribution.
Frequency-Domain	FFT Power Spectrum	Decomposes EEG into frequency components.
	PSD (Welch's Method)	Estimates power spectral density for EEG signals.
	Delta Power (0.5–4 Hz)	Brain state during deep sleep and unconsciousness.

The above table 3.2 illustrates various EEG feature extraction methods categorized into time-domain, frequency-domain, and time-frequency features, each providing valuable insights into brain activity. Time-domain features

analyse the amplitude variations of the EEG signal over time. The mean amplitude represents the average signal value, while variance measures the intensity of signal fluctuations. Skewness and kurtosis describe the shape of the EEG signal distribution, helping to understand data spread and detect extreme values.

5. Model Selection

The extracted features serve as the foundation for selecting a suitable machine learning model for EEG classification. The choice of model depends on various factors, including the complexity of EEG signal patterns, the size of the dataset, and the desired level of accuracy. To achieve optimal performance, model selection involves rigorous evaluation processes such as hyperparameter tuning, cross-validation, and performance metric analysis. Different models are assessed based on their ability to capture temporal and spectral characteristics of EEG signals while maintaining generalization across different subjects and conditions. The final selection is guided by a balance between computational efficiency and classification accuracy, ensuring reliable and interpretable results.

Table 3.3: Models used for classification

Model	Model Decryption	
	Model Type	Model features
Logistic Regression (LR)	ML	Establishes a logistic relationship between EEG features and classification output.
Random Forest (RF)	ML	An ensemble method that builds multiple decision trees for robust classification.
Support Vector Machine (SVM)	ML	Maps EEG features into higher-dimensional space to improve separation between classes.
XGBoost	ML	XGBoost is a fast, accurate model that predicts EEG-based conditions by learning patterns from brain signals.

The above table 3.3 provides an overview of commonly used machine learning models for EEG signal classification. Each model employs a distinct approach to analyse EEG features and enhance classification accuracy.

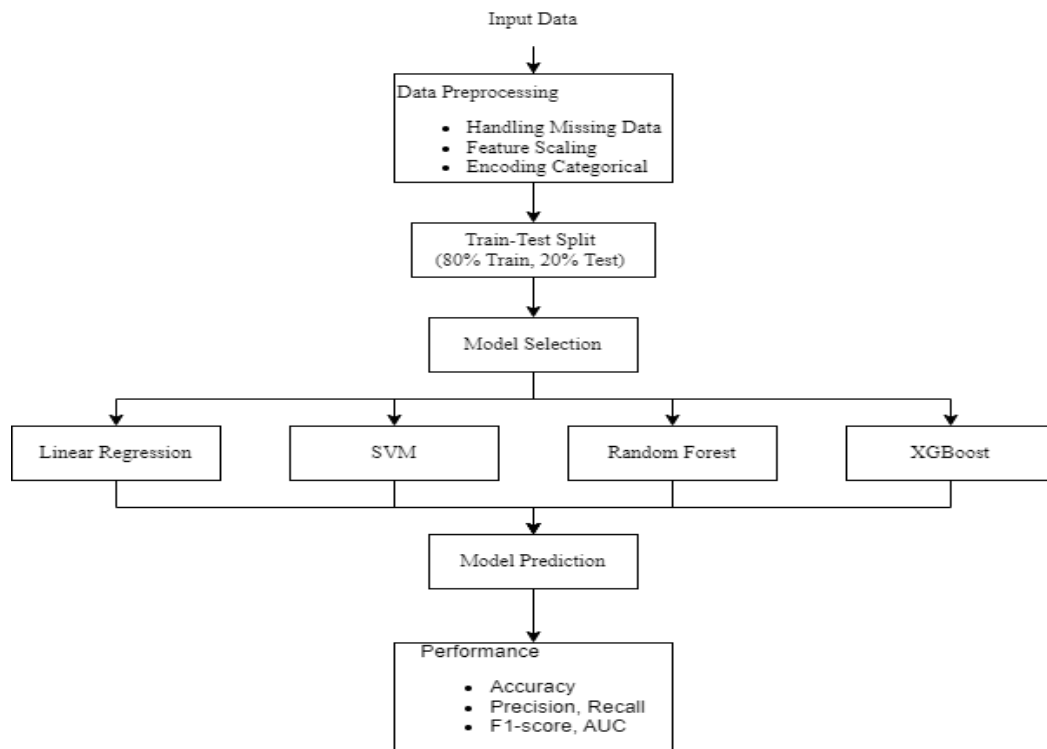


Fig no. 3.4 Model Architecture

In the above Fig 3.4, four machine learning models—Logistic Regression (LR), Support Vector Machine (SVM) and Random Forest (RF) were utilized to perform classification tasks. Each model employs a distinct methodology to learn from the data and generate predictions.

Logistic Regression (LR) is a linear classification algorithm that computes the optimal decision boundary by calculating a weighted sum of the input features. It applies the sigmoid function to transform the output into a probability between 0 and 1, classifying instances based on a predefined threshold (typically 0.5). Owing to its simplicity and interpretability, Logistic Regression is widely used for binary classification problems, particularly when the data exhibits a linear relationship.

Support Vector Machine (SVM) is a kernel-based algorithm designed to identify the optimal hyperplane that effectively separates data points from different classes. Unlike Logistic Regression, which relies on a linear boundary, SVM maximizes the margin between the nearest data points of each class (known as support vectors) to improve generalization. Additionally, SVM incorporates kernel functions (such as Radial Basis Function (RBF) and polynomial kernels) to handle non-linear data by mapping it into a higher-dimensional space, making linear separation possible.

XGBoost (Extreme Gradient Boosting) is an advanced boosting algorithm that builds an ensemble of decision trees sequentially, where each new tree focuses on correcting the errors made by the previous ones. It uses gradient boosting techniques to minimize loss functions efficiently, offering high performance and scalability.

XGBoost incorporates regularization to reduce overfitting and can handle missing data and complex feature interactions, making it particularly effective for large and noisy datasets.

Random Forest (RF) is an ensemble learning technique that constructs multiple decision trees using random subsets of the training data through bootstrap aggregation (bagging). Each decision tree is trained independently, and the final prediction is determined by aggregating the outputs, either through majority voting (for classification) or averaging (for regression). Random Forest improves robustness, reduces variance, and minimizes the risk of overfitting compared to individual decision trees, making it suitable for complex datasets with high dimensionality.

All models in this project were trained on the dataset and evaluated using various performance metrics, including accuracy, precision, recall, F1-score, and AUC-ROC, to assess their effectiveness in classification tasks. Each model offers distinct advantages depending on the dataset's characteristics, computational efficiency, and interpretability, making them applicable across a wide range of predictive modelling scenarios.

Logistic Regression (LR) is a statistical model that establishes a logistic relationship between EEG features and classification output. It is particularly useful for binary classification tasks where the goal is to predict the probability of a specific brain state or condition based on extracted features.

Random Forest (RF) is an ensemble learning method that constructs multiple decision trees and combines their predictions to improve robustness and accuracy. By aggregating outputs from various trees, it reduces overfitting and enhances generalization, making it well-suited for EEG classification tasks where complex patterns need to be recognized.

Support Vector Machine (SVM) is a machine learning algorithm that maps EEG features into a higher-dimensional space to maximize the separation between different classes. By using kernel functions, SVM effectively distinguishes complex EEG patterns, making it a powerful tool for classifying mental states and cognitive processes.

XGBoost (Extreme Gradient Boosting) is a powerful ensemble learning algorithm that builds sequential decision trees, where each tree corrects the errors of the previous ones. It efficiently handles high-dimensional EEG features, captures complex relationships, and provides accurate classification of brain states while minimizing overfitting through regularization techniques.

6. Model Training and Storage

Once a suitable model is chosen, it is trained using labelled EEG datasets. Training involves feeding the model with input data and optimizing its parameters to minimize classification or prediction errors. After successful training, the model is stored for later use in real-time applications. The trained model is saved in a structured format (e.g., TensorFlow Saved Model, PyTorch model format) to enable easy deployment and integration into the prediction pipeline.

To facilitate model deployment and future inference, the trained models were serialized and stored using the pickle library. This allows the models to be loaded later for real-time predictions in the React-based frontend application without requiring retraining. Storing models ensures that the same trained classifiers can be used consistently, maintaining performance and reliability.

Each trained model was saved as a .pkl file using `pickle.dump()`, allowing for quick retrieval and deployment. Additionally, the scaler used for feature transformation was also stored to ensure consistency in preprocessing new data before inference. The stored model can be reloaded using `pickle.load()`, and new EEG data can be pre-processed and classified using the trained model. This approach enhances efficiency by reducing computational overhead and enabling real-time predictions in the deployed system.

By integrating different ML approaches, the proposed methodology ensures robust classification performance, effectively distinguishing MDD from healthy individuals based on EEG-derived features.

7. Frontend Application (React)

To enhance usability, the final predictions and EEG data insights are integrated into a frontend application developed using React. This application provides a user-friendly interface for clinicians, researchers, and end-users to interact with the system. The interface allows users to upload EEG data, visualize processed signals, review model predictions, and interpret diagnostic insights. A well-designed frontend ensures accessibility and ease of use, improving the overall adoption of the system.

8. Final Prediction

The trained model is used to make predictions on new EEG data. Given a new EEG signal, the system processes it using the trained model to classify or predict outcomes such as the presence of depressive symptoms, cognitive workload, or other neurological conditions. The prediction results can be presented in real-time or stored for further analysis by medical professionals.

3.2 Flow Chart

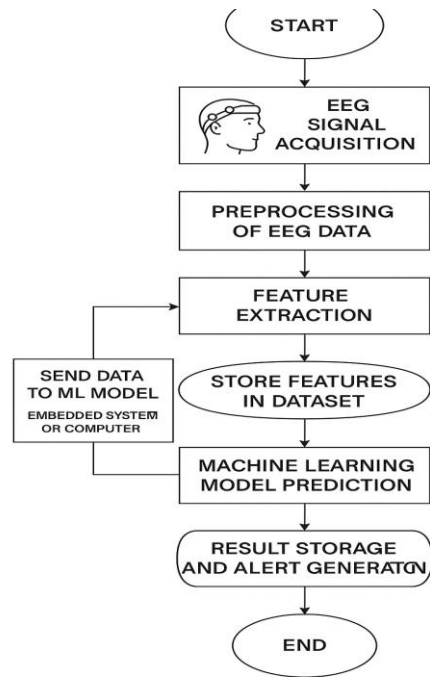


Fig no. 3.5 Flow chart of Proposed System

The diagram outlines a systematic process for improving the early detection of Major Depressive Disorder (MDD) using EEG signals and machine learning. The process begins with the acquisition of real-time EEG signals from subjects through an EEG headset, capturing brainwave activity that reflects mental states.

These raw EEG signals are then sent through a preprocessing stage, where noise, motion artifacts, and other irrelevant signals are removed. This ensures that only clean and usable data proceed to the next step. Once pre-processed, the signals undergo feature extraction, where meaningful characteristics—such as frequency bands, power spectral density, and entropy—are derived to represent the subject's mental condition.

The extracted features are stored in a dataset, forming a structured input for machine learning analysis. An embedded system or a connected computer plays a central role by transmitting this dataset to a trained machine learning model. This model is designed to classify the input data and predict the likelihood of the subject having symptoms of MDD.

Once the prediction is made, the results are stored, and if depressive patterns are detected, an alert is generated to notify medical professionals or connected health systems for further evaluation. This continuous cycle of data acquisition, analysis, and alerting supports early diagnosis and timely intervention.

3.3 Sequence Diagram

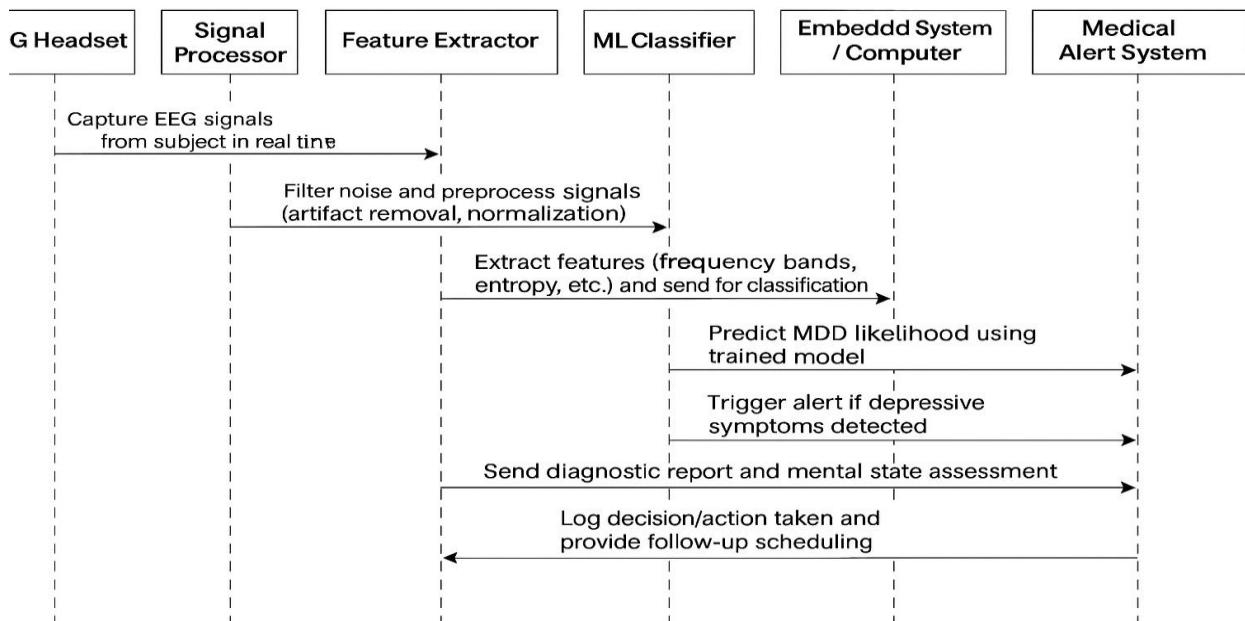


Fig no. 3.6: Sequence Diagram

The given sequence diagram represents an intelligent mental health monitoring system that utilizes EEG signal processing and machine learning to support the early detection of Major Depressive Disorder (MDD). The system comprises several interconnected components working in a coordinated manner to ensure timely and accurate analysis of brain activity.

First, the EEG headset captures real-time brainwave signals from the subject. These raw signals are sent to the Signal Processor, which filters out noise and artifacts (such as muscle movement or electrical interference) to ensure clean data is used for analysis. Once pre-processed, the data is passed to the Feature Extractor, which identifies significant neurological patterns such as frequency bands, entropy, or power spectral density that are indicative of the user's mental state.

These extracted features are then transmitted to the Machine Learning Classifier, which evaluates the input using a pre-trained model designed to recognize signs of depression. Based on this evaluation, the system determines the likelihood of the subject experiencing symptoms of MDD.

If depressive symptoms are detected, the Embedded System or Computer triggers an alert to the Medical Alert System. This ensures that healthcare providers are notified promptly. Simultaneously, a detailed report containing diagnostic insights and mental state assessments is sent to the Clinician Dashboard for review and medical decision-making.

Finally, the clinician logs the action taken, and the system may schedule follow-ups as needed. This intelligent health monitoring system leverages real-time bio signal analysis and AI-based classification to enable proactive mental healthcare, ensuring early intervention, improved outcomes, and continuous patient support.

3.4 ICA component map

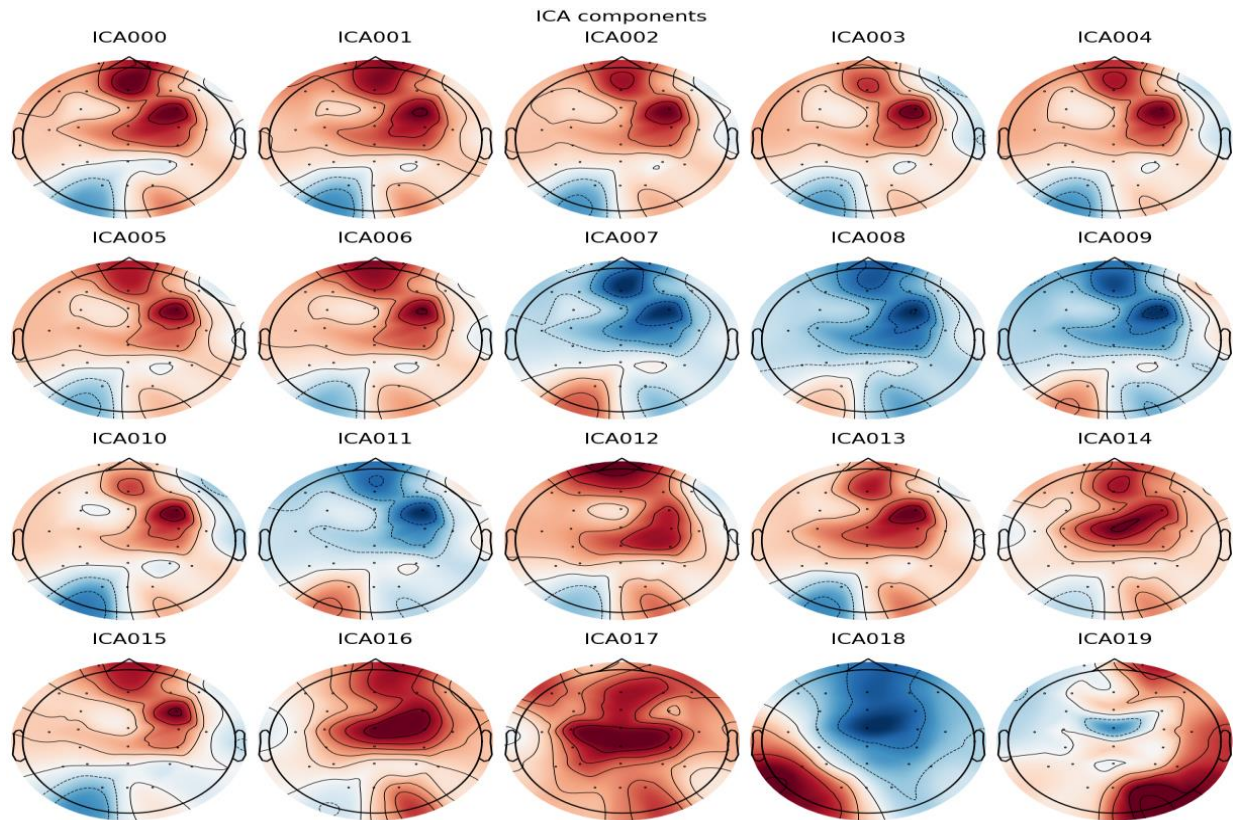


Fig no. 3.7 ICA component map

The above Fig 3.7 provides ICA (Independent Component Analysis) component map represents spatial topographies of EEG signal sources. Each circular plot shows an estimated independent component, with red and blue regions indicating positive and negative activations, respectively. These components help identify and remove artifacts like eye blinks and muscle movements from EEG data.

To mitigate powerline interference, a 50Hz notch filter is applied, followed by a bandpass filter (1–40 Hz) to eliminate low-frequency drifts and high-frequency muscle artifacts.

ICA works by decomposing the EEG signals into statistically independent components. These components are then analyzed to identify those corresponding to artifacts, which are removed before reconstructing the cleaned EEG signals. The identification of artifacts is performed manually or using automated algorithms based on predefined criteria such as spatial distribution, frequency characteristics, and correlation with external reference signals (e.g., EOG for eye movement detection).

3.5 Use Cases

- **Clinical Diagnosis Support:**
 - The system can assist psychologists and psychiatrists by providing objective EEG-based indicators of depressive symptoms, supporting more accurate and early diagnosis.
- **Home-Based Mental Health Tracking:**
 - Individuals can wear EEG headsets at home for continuous monitoring, allowing clinicians to track mental health trends without requiring constant in-person visits.
- **Early Detection in High-Risk Populations:**
 - The system can be used to screen high-risk groups—such as adolescents, postpartum women, or individuals with a family history of depression—for early warning signs before full symptoms develop.
- **Monitoring Treatment Effectiveness:**
 - Clinicians can use EEG data over time to evaluate the effectiveness of prescribed therapies (e.g., CBT, medication), enabling data-driven adjustments to treatment plans.
- **Employee Wellness Monitoring:**
 - Organizations can integrate the system into employee wellness initiatives to identify early stress and depression markers, promoting mental well-being in high-pressure industries
- **Integration with Hospital Systems:**
 - The system can be connected to broader smart healthcare infrastructures, feeding real-time mental health data into electronic health records for comprehensive care management.

CHAPTER 04

SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

- **Development Environment:** Mac OS

- **Code Editors:**

Visual Studio Code – A lightweight but powerful code editor with support for various programming languages and extensions.

- **Frontend Development Tools:**

HTML5: For structuring the web pages.

CSS3: For styling and responsive design.

TypeScript: Adds static typing to JavaScript, helping prevent bugs.

React: A JavaScript library for building user interfaces.

Vite: A fast build tool and development server for modern web projects.

- **Frontend Libraries and Frameworks:**

Recharts: A composable charting library built on React for creating visual data representations.

Lucide-react: For including icons.

Tailwind CSS: A utility-first CSS framework for styling.

ShadCN/UI – For UI components in React.

- **Backend Development Tools:**

Python: Used for backend logic, machine learning model integration, and API development.

Fast API: A modern, fast (high-performance) web framework for building APIs with Python.

Pickle: For loading the trained SVM model (svm_model.pkl).

NumPy & Pandas: For numerical and data manipulation tasks.

- **AI and ML Tools:**

OpenAI's LLM (e.g., GPT models): Used for generating responses, understanding inputs, or performing intelligent analysis.

Pinecone: A vector database used for efficient similarity search in AI tasks.

4.2 Hardware Requirements

- **Development Machine (for coding and testing):**

Processor: Apple M1 chip, macOS Sequoia.

RAM: Minimum 16 GB (32 GB recommended for faster processing during model operations and UI rendering).

Storage: Minimum 512 GB SSD (more if handling large datasets or models).

Display: Full HD (1920x1080) or higher for clear visualization during UI design and debugging



Fig no. 4.1 Python



Fig no. 4.2 Machine Learning



Fig no. 4.3 Vite



Fig no. 4.4 HTML and CSS

CHAPTER 05

IMPLEMENTATION

5.1 Mdd prediction system - App.py

```
import os

import joblib

import NumPy as np

import mne

import pickle

from fast Api import Fast API, File, Upload File, HTTP Exception, Form, Request

from fast Api. responses import JSON Response

from typing import List

from SciPy. Stats import skew, kurtosis

from mne. Preprocessing import ICA

from mne. time frequency import psd_array_welch

import shutil

from flask_cors import CORS

from fastapi.middleware.cors import CORSMiddleware

from fastapi.staticfiles import StaticFiles

import matplotlib.pyplot as plt

import pandas as pd

from scipy.fftpack import fft

import pywt

from sklearn.preprocessing import LabelEncoder

# Initialize FastAPI app

app = FastAPI()

# Add CORS middleware

origins = [

    "http://localhost:5173", # Allow React frontend (change if your frontend runs on a different port)

    "http://localhost:5000", # (optional) Add other frontend URLs here if needed

]
```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allow all origins (or specify allowed origins)
    allow_credentials=True,
    allow_methods=["*"], # Allow all methods (GET, POST, etc.)
    allow_headers=["*"], # Allow all headers
)
# CORS(app)
app.mount("/static", StaticFiles(directory="static"), name="static")
# Directory for storing uploaded files
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
vhdr_path=""
# Load trained model
# MODEL_PATH = "mw/svm_model.pkl"
MODEL_PATH = "model_weights/xgb_model.pkl"
SCALER_PATH = "scaler/xgb_scaler.pkl"
try:
    model = joblib.load(MODEL_PATH) # Load .pkl model
except Exception as e:
    raise RuntimeError(f"Error loading model: {e}")
scaler = joblib.load(SCALER_PATH) # Only keeping scaler for feature scaling
# Store predictions
predictions = []
plot_paths = []
def preprocess_eeg_data(vhdr_file_path, l_freq=1.0, h_freq=40.0, notch_freq=50):
    """Preprocess EEG data."""
    raw = mne.io.read_raw_brainvision(vhdr_file_path, preload=True)
    eog_channels = ['VPVA', 'VNVB', 'HPHL', 'HNHR']
    raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})
    raw.notch_filter(freqs=[notch_freq], picks='eeg')
    raw.filter(l_freq=l_freq, h_freq=h_freq, picks='eeg')

```

```

raw.set_eeg_reference('average', projection=True)

# ICA for artifact removal
ica = mne.preprocessing.ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
eog_indices, _ = ica.find_bads_eog(raw)
ica.exclude = eog_indices
raw = ica.apply(raw)

file_path = os.path.splitext(os.path.basename(vhdr_file_path))[0]
print("**file_path",file_path)
plot_path=f"static/{file_path}.png"
print("**plot_path",plot_path)

fig = raw.plot(scalings={"eeg": 75e-6}, n_channels=32, title="EEG Data", show=True)
fig.savefig(plot_path, dpi=300)
plt.close(fig)

return raw

def extract_channel_features(raw, fmin=0.5, fmax=50):
    # Select only EEG channels
    raw.pick_types(eeg=True) # This removes non-EEG channels
    data = raw.get_data()
    channel_names = raw.ch_names
    features = {ch: {} for ch in channel_names}

# Handle categorical data
for col in df.select_dtypes(include=['object']).columns:
    df[col] = LabelEncoder().fit_transform(df[col])

# Handle missing values
df.fillna(df.median(), inplace=True)

return df

def predict_eeg(eo_path, ec_path):
    # Load and Preprocess the Single CSV File
    outut_file = "preprocessed.csv"
    process_and_combine(eo_path, ec_path, outut_file)
    combined_features = pd.read_csv(outut_file)

```

```

X_predict = load_and_preprocess_single_csv(combined_features)
best_model = joblib.load(MODEL_PATH)
# Apply Same Scaling as Training Data
X_predict_scaled = scaler.transform(X_predict)
# Predict on the Single CSV Data
prediction = best_model.predict(X_predict_scaled)
final_prediction = prediction[0]
try:
    probability = best_model.predict_proba(X_predict_scaled)[0] # Probabilities for both classes
    probability_healthy = probability[0] * 100
    probability_mdd = probability[1] * 100
    print(f"Probability of being Healthy: {probability_healthy:.2f}%")
    print(f"Probability of having MDD: {probability_mdd:.2f}%")
except AttributeError:
    print("Prediction probabilities are not available for this model.")
except Exception as e:
    print(f"Error getting prediction probabilities: {e}")
alpha_power = combined_features["alpha_power"].mean()
beta_power = combined_features["beta_power"].mean()
delta_power = combined_features["delta_power"].mean()
theta_power = combined_features["theta_power"].mean()
final_prediction = "MDD" if final_prediction == 1 else "Healthy"
final_prob = probability_mdd if final_prediction == "MDD" else probability_healthy
return final_prediction, final_prob, alpha_power, beta_power, delta_power, theta_power
def visualize_eeg_data(vhdr_path):
    """Visualize EEG data."""
# Convert to Pandas DataFrame
raw = preprocess_eeg_data(vhdr_path)
df = raw.to_data_frame()

# Save as JSON for React
df.to_json("eeg_data.json", orient="records")

```



```

return df

@app.post("/upload/")
async def upload_files(
    vhdrEO: UploadFile = File(...), vmrkEO: UploadFile = File(...), eegEO: UploadFile = File(...),
    vhdrEC: UploadFile = File(...), vmrkEC: UploadFile = File(...), eegEC: UploadFile = File(...)
):
    file_paths = {}
    # Save files to the upload folder
    for file in [vhdrEO, vmrkEO, eegEO, vhdrEC, vmrkEC, eegEC]:
        file_path = os.path.join(UPLOAD_FOLDER, file.filename)
        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)
        file_paths[file.filename] = file_path # Store file paths
    print("Files uploaded successfully:", file_paths)
    return {"message": "Files uploaded successfully", "file_paths": file_paths}

```

5.2 Mdd prediction system – model_prediction. ipynb

```

import joblib

import numpy as np

import mne

import numpy as np

from scipy.stats import skew, kurtosis

from mne.preprocessing import ICA

from mne.time_frequency import psd_array_welch

import pandas as pd

from scipy.fftpack import fft

import pywt # For wavelet transform

def preprocess_eeg_data(vhdr_file_path, l_freq=1.0, h_freq=40.0, notch_freq=50):
    raw = mne.io.read_raw_brainvision(vhdr_file_path, preload=True)

    eog_channels = ['VPVA', 'VNVB', 'HPHL', 'HNHR']

    raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})

    raw.notch_filter(freqs=[notch_freq], picks='eeg')

```

```

raw.filter(l_freq=l_freq, h_freq=h_freq, picks='eeg')
raw.set_eeg_reference('average', projection=True)
# ICA for artifact removal
ica = ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
eog_indices, _ = ica.find_bads_eog(raw)
ica.exclude = eog_indices
raw = ica.apply(raw)
return raw

def extract_channel_features(raw, fmin=0.5, fmax=50):
    # Select only EEG channels
    raw.pick_types(eeg=True) # This removes non-EEG channels
    data = raw.get_data()
    channel_names = raw.ch_names
    features = {ch: {} for ch in channel_names}
    # Time-domain features
    for i, ch in enumerate(channel_names):
        features[ch]['mean'] = np.mean(data[i])
        features[ch]['variance'] = np.var(data[i])
        features[ch]['skewness'] = skew(data[i])
        features[ch]['kurtosis'] = kurtosis(data[i])
        features[ch]['peak_to_peak'] = np.ptp(data[i])
    # Fourier Transform (FFT)
    fft_values = np.abs(fft(data[i]))
    features[ch]['fft_mean'] = np.mean(fft_values)
    features[ch]['fft_std'] = np.std(fft_values)
    features[ch]['fft_max'] = np.max(fft_values)
    # Wavelet Transform (DWT) using Daubechies wavelet (db4) #morle
    coeffs = pywt.wavedec(data[i], 'db4', level=4)
    features[ch]['wavelet_energy'] = sum(np.sum(np.square(c)) for c in coeffs)
    features[ch]['wavelet_entropy'] = 0 # Initialize wavelet_entropy
    for c in coeffs:

```

```

c = c[np.isfinite(c)]
c_norm = c / (np.sum(np.abs(c)) + 1e-10)
features[ch]['wavelet_entropy'] += -np.sum(c_norm * np.log2(c_norm + 1e-10))

# Frequency-domain features using PSD
psd = raw.compute_psd(method='welch', fmin=fmin, fmax=fmax, n_fft=2048)
psd_data = psd.get_data()
freqs = psd.freqs
psd_df = pd.DataFrame(psd_data, columns=freqs, index=channel_names)
bands = {'delta': (0.5, 4), 'theta': (4, 8), 'slow_alpha': (6, 9), 'alpha': (8, 12),
         'beta': (12, 30), 'gamma': (30, 50)}
for band, (low, high) in bands.items():
    band_power = psd_df.loc[:, (freqs >= low) & (freqs <= high)].mean(axis=1)
    for ch in channel_names:
        features[ch][f'{band}_power'] = band_power[ch]

# Frontal Alpha Asymmetry (F3-F4)
if 'F3' in channel_names and 'F4' in channel_names:
    features['F3_F4_alpha_asymmetry'] = features['F4']['alpha_power'] - features['F3']['alpha_power']

# Convert features to DataFrame
features_df = pd.DataFrame(features).T
return features_df

def process_and_combine(eo_file_path, ec_file_path, output_file):
    all_features = []

    # Process EO file
    raw_eo = preprocess_eeg_data(eo_file_path)
    features_eo = extract_channel_features(raw_eo)
    #features_eo['condition'] = 'EO'
    all_features.append(features_eo)

    # Process EC file
    raw_ec = preprocess_eeg_data(ec_file_path)
    features_ec = extract_channel_features(raw_ec)
    #features_ec['condition'] = 'EC'
    all_features.append(features_ec)

```

```
# Combine EO and EC features
combined_features = pd.concat(all_features, keys=['EO', 'EC'], names=['condition', 'channel'])

# Save combined features to a single CSV file
combined_features.to_csv(output_file)

print(f"Features successfully saved to {output_file}")

# return combined_features

import pandas as pd
import joblib
from sklearn.preprocessing import LabelEncoder

# Modified Function to Load and Preprocess a Single CSV File
def load_and_preprocess_single_csv(df):
    Loads and preprocesses a single CSV file for prediction.

    Args:
        file_path (str): The path to the CSV file.

    Returns:
        pandas.DataFrame: Preprocessed DataFrame.

    # Handle categorical data
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = LabelEncoder().fit_transform(df[col])

    # Handle missing values
    df.fillna(df.median(), inplace=True)

    return df

def predict(eo_path, ec_path):
    # Load and Preprocess the Single CSV File
    output_file = "preprocessed.csv"
    process_and_combine(eo_path, ec_path, output_file)
    csv_data = pd.read_csv(output_file)
    X_predict = load_and_preprocess_single_csv(csv_data)

    # Load Pre-trained Model and Scaler
    best_model = joblib.load("../model_weights/xgb_model.pkl")
    scaler = joblib.load("../scaler/xgb_scaler.pkl") # Load the same scaler used during training
```

```

# Apply Same Scaling as Training Data
X_predict_scaled = scaler.transform(X_predict)

# Predict on the Single CSV Data
prediction = best_model.predict(X_predict_scaled)
predicted_class = prediction[0]

if predicted_class == 0:
    prediction_label = "Healthy (0)"
elif predicted_class == 1:
    prediction_label = "MDD (1)"
else:
    prediction_label = "Unknown" # Should not happen in binary classification, but for robustness
print(f"Class: {predicted_class} - {prediction_label}")

try:
    probability = best_model.predict_proba(X_predict_scaled)[0] # Probabilities for both classes
    probability_healthy = probability[0] * 100
    probability_mdd = probability[1] * 100
    print(f"Probability of being Healthy: {probability_healthy:.2f}%")
    print(f"Probability of having MDD: {probability_mdd:.2f}%")
except AttributeError:
    print("Prediction probabilities are not available for this model.")
except Exception as e:
    print(f"Error getting prediction probabilities: {e}")

```

5.3 Mdd prediction system – perprocessing.ipynb

```

import mne
import numpy as np
import matplotlib.pyplot as plt
import os

from mne.preprocessing import ICA

raw = mne.io.read_raw_brainvision('path_to_vhdr_file',preload=True)

raw.plot(show=True)

raw.filter(l_freq=0, h_freq=40) # Bandpass filter

```

```

eog_channels = ['VPVA', 'VNVB', 'HPLH', 'HNHR','Erbs','OrbOcc', 'Mass']
raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})
montage = mne.channels.make_standard_montage('standard_1020')
raw.set_montage(montage,on_missing='ignore')
ica = ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
sources = ica.get_sources(raw).get_data()
ica.plot_components()
raw.plot(show=True)

```

5.4 Algorithm

Table 5.1 describes the performance of different machine learning models evaluated on the dataset using key classification metrics: accuracy, precision, recall, F1-score, and AUC-ROC. These metrics provide insights into how well each model performs in classifying the data.

TABLE 5.1: Comparison Of Performance Measure

Model	Performance Measure				
	Accuracy (%)	Precision	Recall	F1-score	AUC-ROC
Logistic Regression (LR)	62.14	0.62	0.62	0.61	0.68
Random Forest (RF)	79.17	0.79	0.79	0.79	0.88
Support Vector Machine (SVM)	66.30	0.66	0.66	0.63	0.73
XGBoost	96.00	0.96	0.95	0.95	0.90

Logistic Regression achieved an accuracy of 0.6214, with a precision of 0.6267, recall of 0.6214, and an F1-score of 0.6191. This indicates moderate performance but may struggle with complex decision boundaries.

The XGBoost model outperformed the other models, achieving the highest accuracy of 0.96, along with strong precision (0.96), recall (0.98), and F1-score (0.90). This suggests that XGBoost effectively captures patterns in the data and provides better classification performance.

The Random Forest model showed moderate performance, achieving the accuracy of 0.7917, along with precision (0.7917), recall (0.7917), and F1-score (0.7917).

The Support Vector Machine (SVM) model showed moderate performance, with an accuracy of 0.6630, precision of 0.6695, recall of 0.6630, and an F1-score of 0.6610. While better than Logistic Regression, it does not perform as well as Random Forest.

```

Model: Logistic Regression
Accuracy: 0.6214

```

	precision	recall	f1-score	support
0	0.655914	0.541741	0.593385	563.000000
1	0.596244	0.704251	0.645763	541.000000
accuracy	0.621377	0.621377	0.621377	0.621377
macro avg	0.626079	0.622996	0.619574	1104.000000
weighted avg	0.626674	0.621377	0.619052	1104.000000

```

=====

```

Fig no. 5.1: Performance metrics of Logistic regression

Fig 5.1 illustrates the classification performance of the Logistic Regression model, which achieves an accuracy of 62.14%. The model's precision and recall scores indicate an imbalance between the two classes. Class 0 exhibits higher precision, meaning it correctly identifies non-MDD cases while minimizing false positives. In contrast, Class 1 has better recall, suggesting that the model captures more actual MDD cases but at the cost of increased false positives.

```

=====
Model: Random Forest
Accuracy: 0.7917

```

	precision	recall	f1-score	support
0	0.797853	0.792185	0.795009	563.000000
1	0.785321	0.791128	0.788214	541.000000
accuracy	0.791667	0.791667	0.791667	0.791667
macro avg	0.791587	0.791656	0.791611	1104.000000
weighted avg	0.791712	0.791667	0.791679	1104.000000

```

=====

```

Fig no.5.2: Performance metrics of Random Forest

Fig 5.2 illustrates the classification performance of the Random Forest model, which achieves an accuracy of 79.17%. The precision and recall scores reveal a balanced performance between the two classes. Class 0 demonstrates slightly higher precision, effectively minimizing false positives, while Class 1 maintains a comparable recall, ensuring that actual positive cases are well captured.

```

=====
Model: SVM
Accuracy: 0.6630

```

	precision	recall	f1-score	support
0	0.705376	0.582593	0.638132	563.000000
1	0.632238	0.746765	0.684746	541.000000
accuracy	0.663043	0.663043	0.663043	0.663043
macro avg	0.668807	0.664679	0.661439	1104.000000
weighted avg	0.669536	0.663043	0.660975	1104.000000

```

=====

```

Fig no.5.3: Performance metrics of SVM

Fig 5.3 illustrates the performance of the SVM model, as shown in the figure, achieves an accuracy of 66.30%. The precision score is higher for class 0, indicating that the model effectively minimizes false positives for non-MDD cases. Meanwhile, class 1 exhibits better recall, suggesting that the model captures more actual MDD cases but at the cost of increased false positives.

```

🔥 Validation Accuracy: 0.9550

```

♦ Classification Report:

	precision	recall	f1-score	support
0	0.96	0.94	0.95	2760
1	0.95	0.97	0.96	3600
accuracy			0.96	6360
macro avg	0.96	0.95	0.95	6360
weighted avg	0.96	0.96	0.95	6360

Fig no. 5.4 : Performance metrics of XGBoost

Fig 5.4 illustrates the performance of the XGBoost model, which achieves a high accuracy of 95.50%. The precision score is slightly higher for class 0, indicating effective minimization of false positives for non-MDD cases. On the other hand, class 1 demonstrates superior recall, highlighting the model's ability to accurately identify most actual MDD cases, making XGBoost highly reliable for EEG-based MDD classification with balanced precision and recall across both classes.

TESTING

6.1 Project Testing

- **Core Functionality:** The application detects Major Depressive Disorder (MDD) in patients using EEG signal features and machine learning models. It provides a Stream lit-based user interface where users can upload EEG feature data or enter specific signal metrics. The system returns a prediction indicating the likelihood of MDD.
- **Model Integration and Input Handling:** Pre-trained machine learning models (e.g., SVM, Random Forest, or Neural Networks) are loaded using the pickle module. Inputs such as EEG frequency bands (alpha, beta, theta, delta) and derived features (e.g., entropy, asymmetry indices) are accepted via structured forms. Input validation is limited and requires improvement to handle missing, incorrectly formatted, or out-of-range values
- **Testing Strategies:**
 - **Unit testing:** Validates the loading of models, correct parsing and processing of EEG features, and consistency of predictions
 - **Integration Testing:** Ensures end-to-end flow from EEG data input to prediction output, including interaction between frontend (stream lit) and backend (ML model).
- **Performance and Error Handling:**
 - **Performance testing:** Measures prediction latency and verifies the system can handle multiple prediction requests concurrently without degradation
 - **Error Handling:** Simulates missing or corrupted model files, incorrect EEG input formats, and system exceptions to ensure meaningful feedback is shown and the app doesn't crash
- **User Experience:** Evaluates the clarity and relevance of EEG input labels, the helpfulness of tooltips or help texts, and ease of navigation in the interface. Testing highlights areas needing improvement in input validation, error messages, and overall accessibility for users without a technical background
- **Performance and Scalability:** If using cloud resources or external EEG feature processing APIs, assess their response under stress. Ensure the feature extraction module scales well with batch EEG inputs or streaming data
- **User Experience:** Test how intuitive the EEG data upload or manual entry process is Check system feedback for incorrect or low-quality EEG input data.

6.2 Dataset of healthys.csv for Testing

These are the attributes used to predict the health of a patient.

name	type	units	low_cutoff		high_cutoff	description	sampling_frequency	
Fp1	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Fp2	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
F7	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
F3	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Fz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
F4	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
F8	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
FC3	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
FCz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
FC4	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
T7	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
C3	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Cz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
C4	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
T8	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
CP3	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
CPz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
CP4	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
P7	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
P3	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Pz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
P4	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
P8	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
O1	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Oz	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
O2	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
VPVA	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
VNVB	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
HPHL	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
HNHR	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Erbs	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
OrbOcc	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a
Macc	EEG	V	0.0	250.0	ElectroEncephaloGram	500.0	good	n/a

Fig no. 6.1 Health dataset

RESULTS

7.1 Snapshots of Project

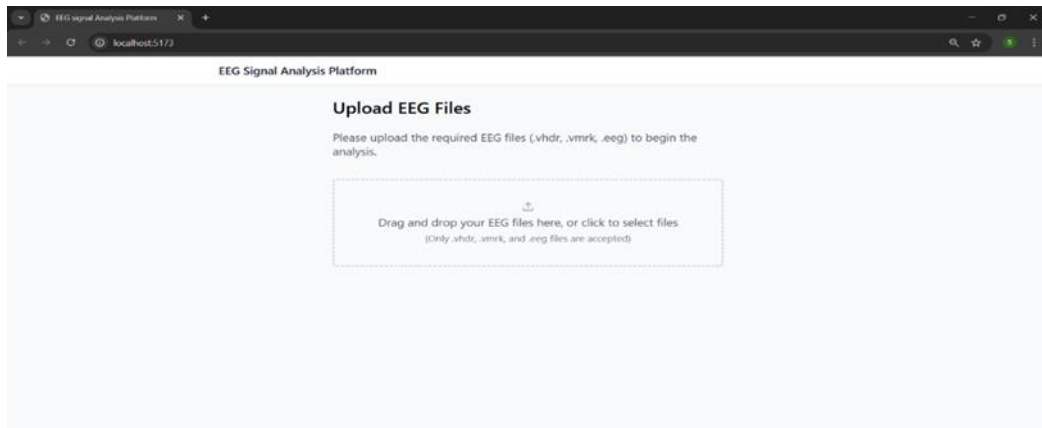


Fig no.7.1: User Interface

The above fig 7.1 provides the frontend of EEG Signal Analysis Platform is designed to provide a user-friendly interface for interacting with the system. It facilitates key functionalities such as file uploading, user interaction, data visualization, and result interaction. The platform allows users to upload EEG data files (.vhdr, .vmrk, .eeg) either by dragging and dropping them into a designated area or selecting them through a file browser, ensuring validation of file extensions for supported formats.

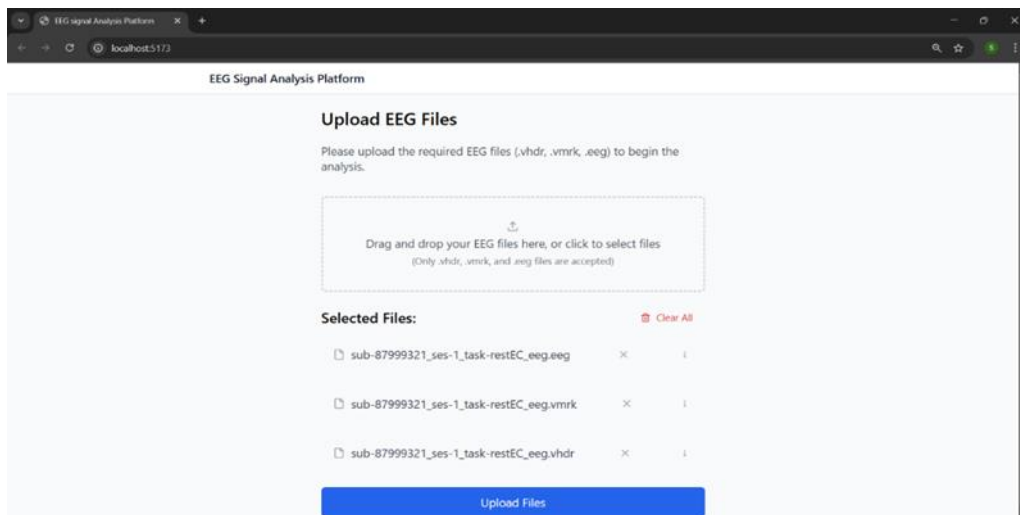


Fig no.7.2: File Upload Interface

The above fig 7.2 illustrates EEG Signal Analysis Platform's frontend provides a user-friendly mechanism for uploading EEG data files, allowing users to choose between drag-and-drop or manual file selection. In the drag-and-drop method, users can simply drag .vhdr, .vmrk, and .eeg files onto the designated drop area, which typically includes visual cues like a highlighted box or a "Drop files here" message for guidance.

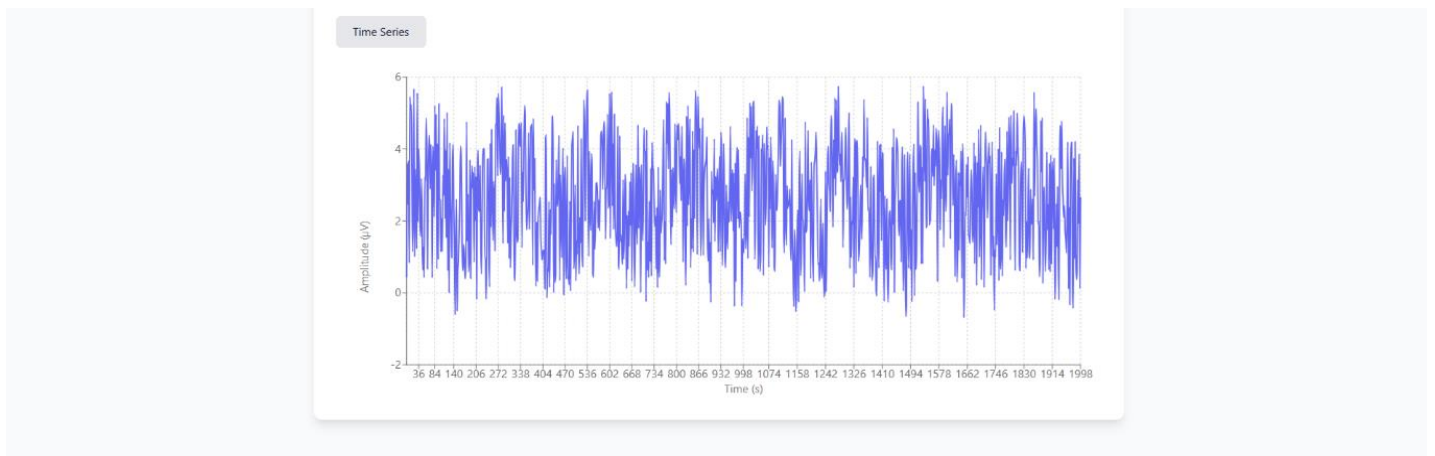
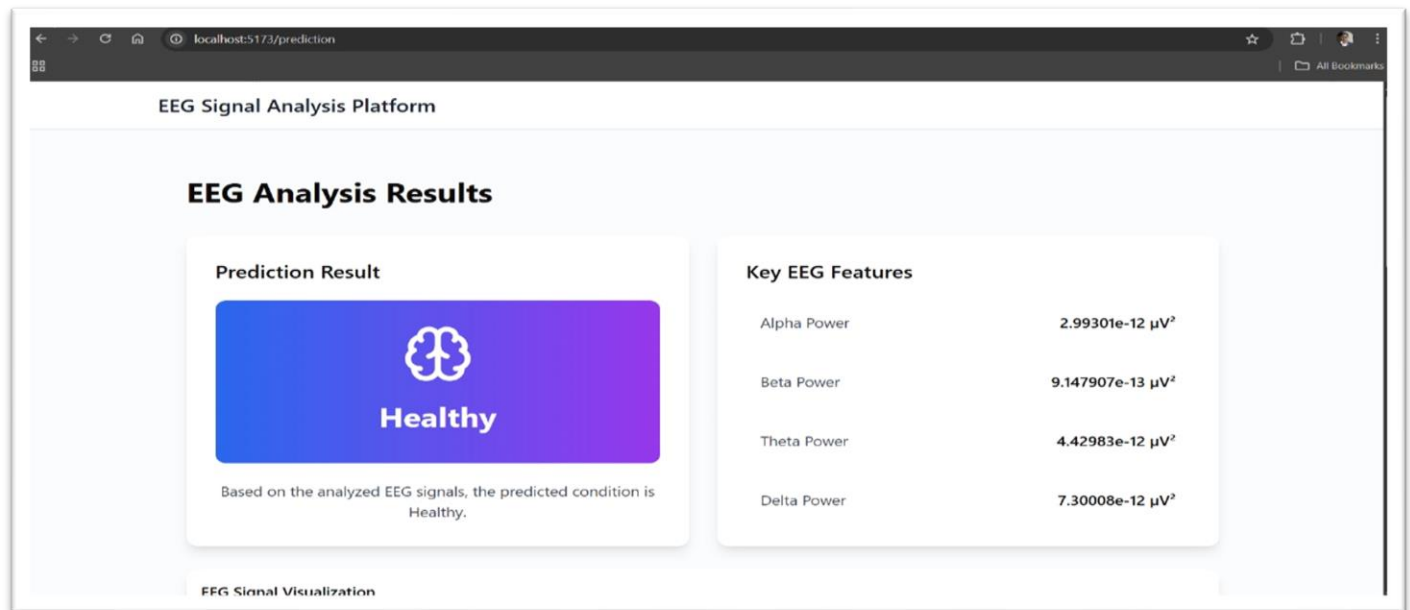


Fig no.7.3 Predicting Person as healthy

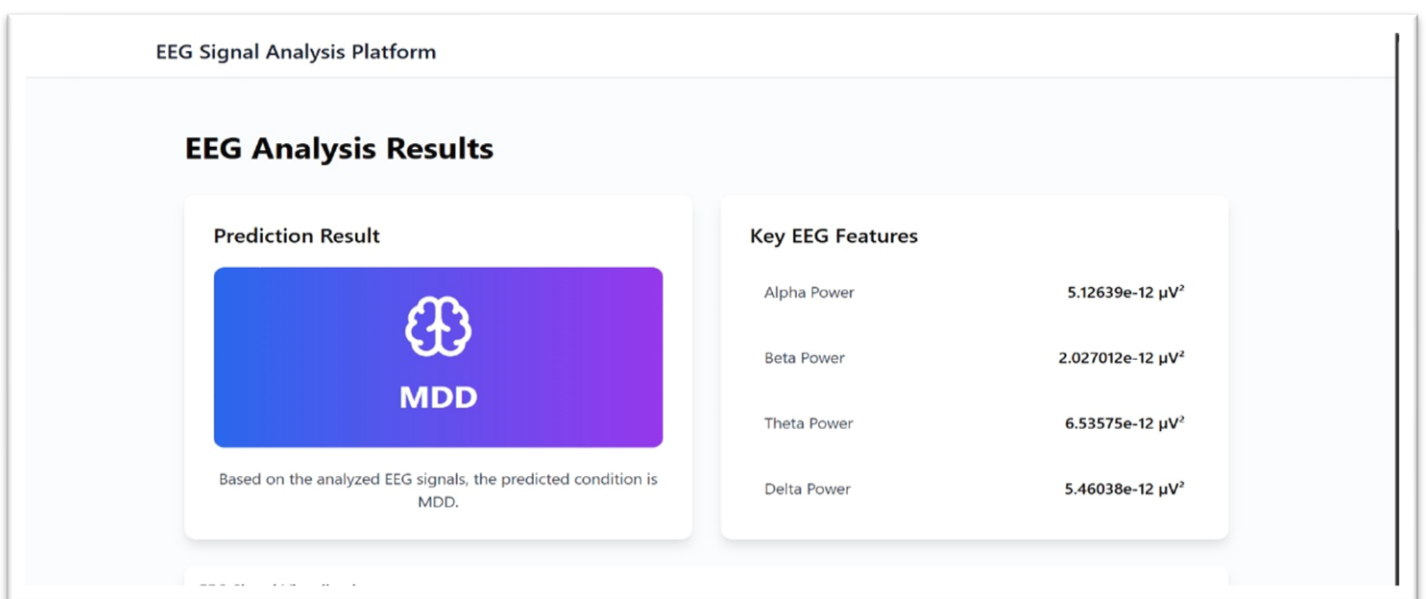


Fig no.7.4 Predicting Person as MDD

CONCLUSION

The study on EEG-based classification for early detection of Major Depressive Disorder (MDD) highlights the effectiveness of machine learning techniques in analysing neural activity and identifying patterns associated with mental health conditions. The project involved a systematic workflow, starting from EEG data collection and preprocessing to feature extraction, model selection, and classification. Each step played a crucial role in enhancing the accuracy and reliability of the final predictions.

Preprocessing proved to be a vital step in improving signal quality by removing artifacts, noise, and unwanted interference. The application of filters such as notch filtering, bandpass filtering, and Independent Component Analysis (ICA) helped in preserving meaningful neural activity while eliminating external disturbances. This ensured that the extracted features were representative of actual brain dynamics rather than artifacts introduced during data collection.

Feature extraction focused on identifying key attributes of the EEG signals that contribute to distinguishing different mental states. By utilizing time-domain, frequency-domain, and time-frequency features, the extracted parameters provided a comprehensive representation of neural activity. Frequency-specific features, such as power in delta, theta, alpha, beta, and gamma bands, played a crucial role in understanding cognitive states, while statistical measures such as variance, skewness, and kurtosis contributed to signal characterization.

Model selection and classification involved evaluating multiple approaches to identify the most effective learning framework for EEG-based MDD detection. The choice of model depended on factors such as feature complexity, dataset size, and classification accuracy. To ensure robust generalization, techniques like cross-validation and hyperparameter tuning were applied, optimizing the model's ability to distinguish between healthy individuals and those with MDD.

The results demonstrated that machine learning models can successfully classify EEG signals related to depressive symptoms, reinforcing the potential of EEG-based biomarkers in mental health diagnostics. However, challenges remain, including inter-subject variability, dataset limitations, and the need for real-time adaptability in practical applications. Future advancements may focus on improving model interpretability, integrating deep learning.

architectures for more complex feature representations and developing personalized diagnostic systems that adapt to individual brain patterns.

FUTURE SCOPE

1. Expansion of Mental Health Disorder Detection

- Extend the system's capabilities to detect other mental health conditions such as anxiety, bipolar disorder, and PTSD by training on multi-disorder EEG datasets.

2. Enhanced EEG Signal Preprocessing

- Integrate advanced noise filtering and artifact removal techniques to improve signal clarity and model accuracy, especially in non-clinical environments.

3. Multimodal Data Integration

- Combine EEG data with other biomarkers (e.g., heart rate variability, facial expressions, speech patterns) for more robust and holistic assessments.

4. Remote Data Collection and Telepsychiatry Integration

- Enable integration with wearable EEG devices and telepsychiatry platforms to support remote diagnosis and consultations.

5. Personalized Depression Risk Profiling

- Develop user-specific profiles that consider individual baseline EEG patterns, lifestyle, and history to provide customized risk assessments

6. Real-Time Monitoring and Alerts

- Implement real-time EEG data processing and alert systems to notify users or caregivers of early depressive symptoms as they occur.

7. Continuous Learning and Model Adaptation

- Use reinforcement learning or continual learning algorithms to update the model based on new user data and evolving patterns in mental health.

8. Self-Help Recommendations and Mental Wellness Tips

- Incorporate evidence-based self-help strategies and mental wellness advice based on individual predictions and mood patterns.

9. Multilingual and Emotion-Sensitive Feedback System

- Provide user-friendly, emotionally aware feedback in multiple languages to support diverse populations and reduce stigma in mental health care.

10. Mobile and Cloud-Based Platform Development

- Develop a cross-platform mobile and cloud-based solution to enable easy access and storage of EEG data and prediction results, enhancing usability and reach

REFERENCE

- [1] Zhou, T., Zhang, Z., & Zhang, Q. (2023). Electroencephalography-based depression detection using multiple machine learning techniques. *Diagnostics*, 13(10), 1779
- [2] Lin, H., Fang, J., Zhang, J., Zhang, X., Piao, W., & Liu, Y. (2024). Resting-State Electroencephalogram Depression Diagnosis Based on Traditional Machine Learning and Deep Learning: A Comparative Analysis. *Sensors*, 24(21), 6815.
- [3] Khadidos, A. O., Alyoubi, K. H., Mahato, S., Khadidos, A. O., & Mohanty, S. N. (2023). Computer-Aided Detection of Major Depressive Disorder (MDD) Using Electroencephalogram Signals. *IEEE Access*, 11, 41133-41141.
- [4] Wang, Y., Zhao, S., Jiang, H., Li, S., Luo, B., Li, T., & Pan, G. (2024). DiffMDD: A Diffusion-Based Deep Learning Framework for MDD Diagnosis Using EEG. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*.
- [5] Li, X., Zhang, X., Zhu, J., Mao, W., Sun, S., Wang, Z., Xia, C., & Hu, B. (2019). Depression recognition using machine learning methods with different feature generation strategies. *Artificial Intelligence in Medicine*, 99, 101696.
- [6] Xia, M., Zhang, Y., Wu, Y., & Wang, X. (2023). An End-to-End Deep Learning Model for EEG-Based Major Depressive Disorder Classification. *IEEE Access*, 11, 41045-41054.
- [7] Shin, D, Lee, KJ, Adeluwa, T, and Hur, J. Machine learning-based predictive modelling of postpartum depression. *J Clin Med*. (2020) 9:2899. Doi: 10.3390/jcm9092899
- [8] Bzdok, D, Altman, N, and Krzywinski, M. Statistics versus machine learning. *Nat Methods*. (2018) 15:233–4. Doi: 10.1038/nmeth.4642 PubMed